



One calm home for everything you're studying.

Student Productivity & Academic Management Workspace

HTML5

CSS3

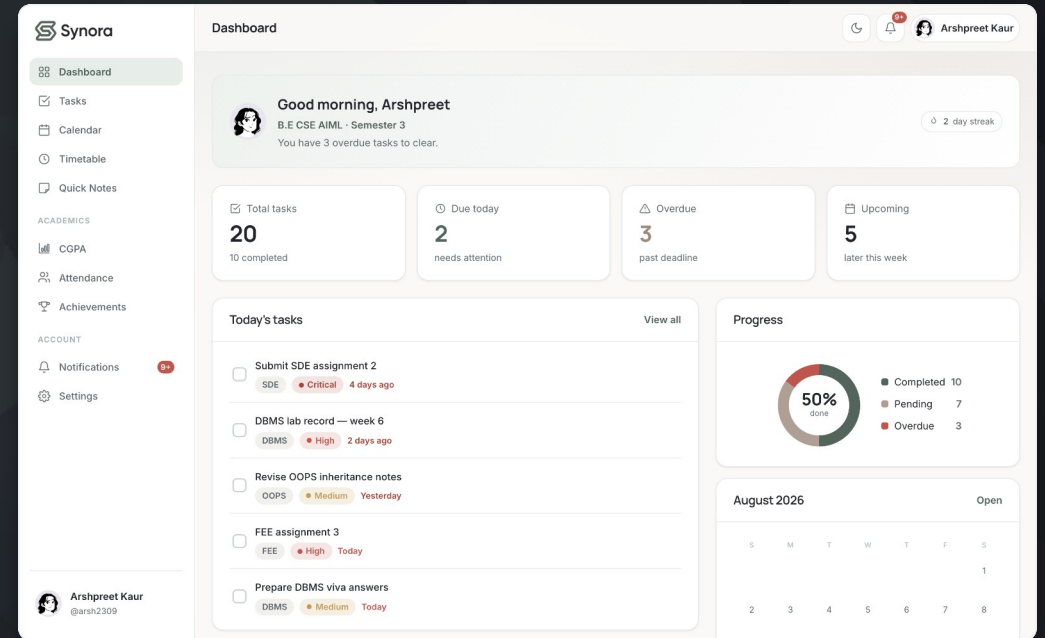
JavaScript

DOM

Events

SUBMITTED BY

Manan Kochhar · Arshpreet Kaur · Atharv Bains



The Synora dashboard — a live, working product.

01 · THE PROBLEM

A student's semester is scattered across apps that never talk.



To-do app



Sticky notes



Attendance sheet



Wall calendar



Reminders



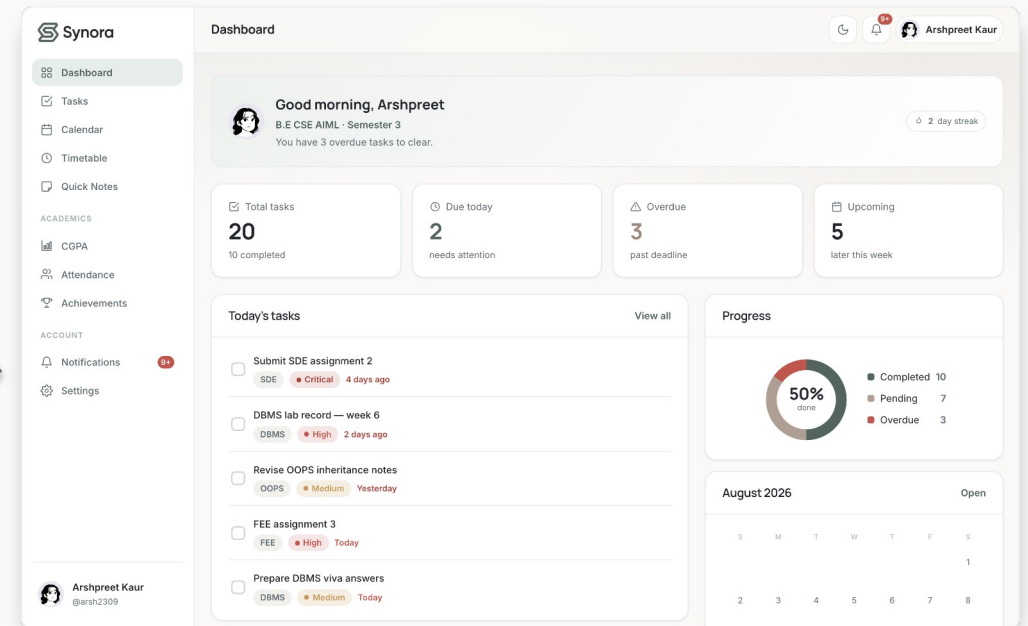
Grades sheet



Six apps. Zero that talk to each other — so deadlines slip through the gaps and nothing sees the whole week.

Eight tools, gathered into one calm workspace.

- Tasks
- Calendar
- Notes
- Timetable
- Attendance
- CGPA
- Achievements
- Notifications



One dashboard for the whole semester.

One account. One quiet home — personal to each student, on every device.

What is Synora?

A personalized student productivity workspace — academic planning, task management and performance tracking in one interface. Its features fall into four groups:



Productivity



Tasks



Calendar



Quick Notes



Academic



Timetable



Attendance



CGPA



Progress



Achievements



Notifications



Personalization



Profile



Subjects



Avatar & Theme

Technology stack

Core web technologies only — no frameworks, no build step.



HTML5

Structure & semantics



CSS3

Design system + responsive layout



JavaScript

Logic, state & interactivity



DOM APIs

A live, dynamic interface



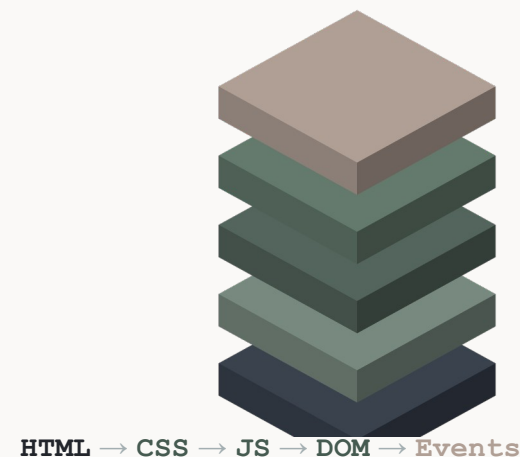
localStorage

Per-user persistence in the browser



Git & GitHub

Version control



How the project is structured

No framework, no build step — files a browser opens directly. One page and one module per feature.

Synora/

└─ index.html	Landing page
└─ notes.html	Project & viva doc
└─ README.md	Documentation
└─ app/	the signed-in workspace
│ └─ dashboard.html	Home overview
│ └─ tasks.html	Task manager
│ └─ cgpa.html	CGPA calculator
│ └─ ... 10 more	timetable, attendance ...
└─ scripts/	18 vanilla-JS modules
│ └─ app.js	Shared core + helpers
│ └─ storage.js	localStorage layer
│ └─ ... 16 more	auth, theme, tasks ...
└─ styles/	7 stylesheets
│ └─ tokens.css	Design tokens
│ └─ ... 6 more	base, components ...
└─ assets/	logos · avatars · icons

14

HTML pages

7

CSS files

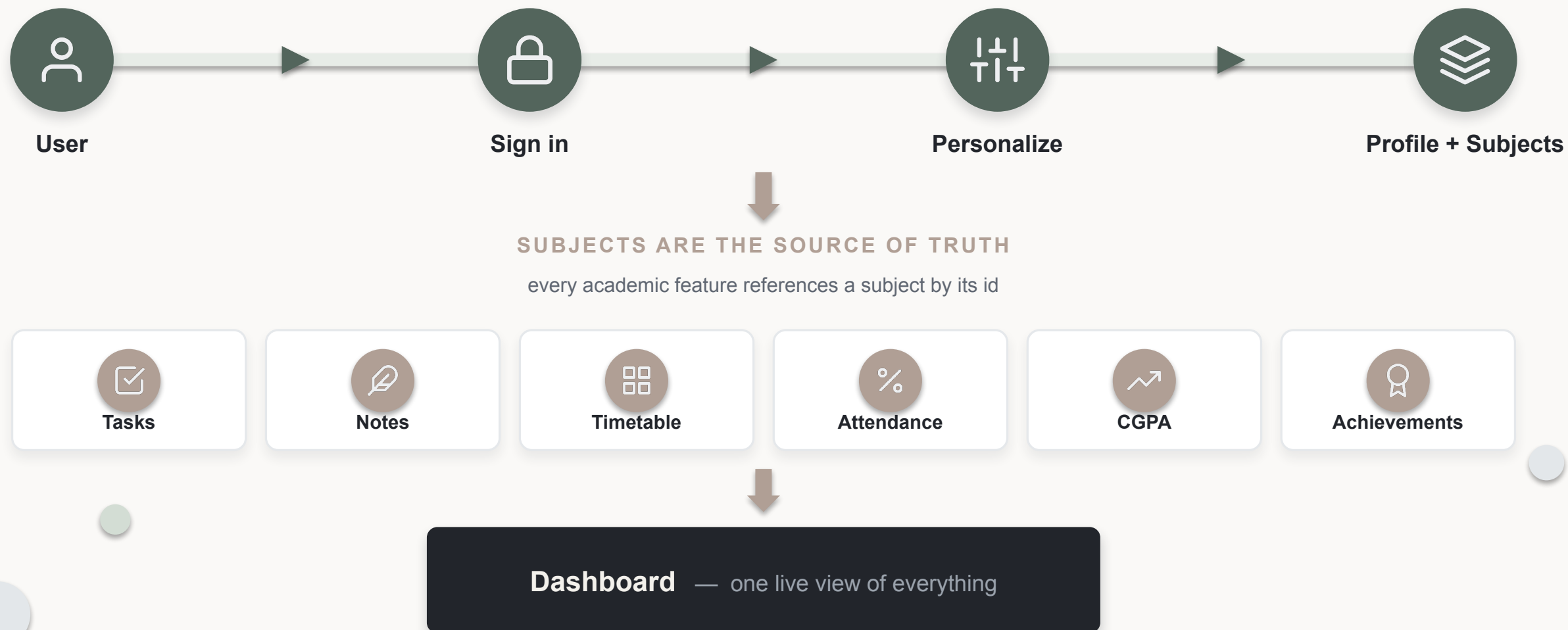
18

JS modules

0

frameworks

Data flow



The structure behind Synora

Every page is written with semantic HTML5 — elements that describe what each region means, not just how it looks. The browser, assistive tech and our own JavaScript all read that same structure.

- <> **<header>** site header + nav
- <> **<main>** ×14 primary content
- <> **<section>** ×11 labelled regions
- <> **<aside>** ×12 sidebar
- <> **<footer>** landing footer
- <> **<figure>** ×3 media

```
index.html — landing structure

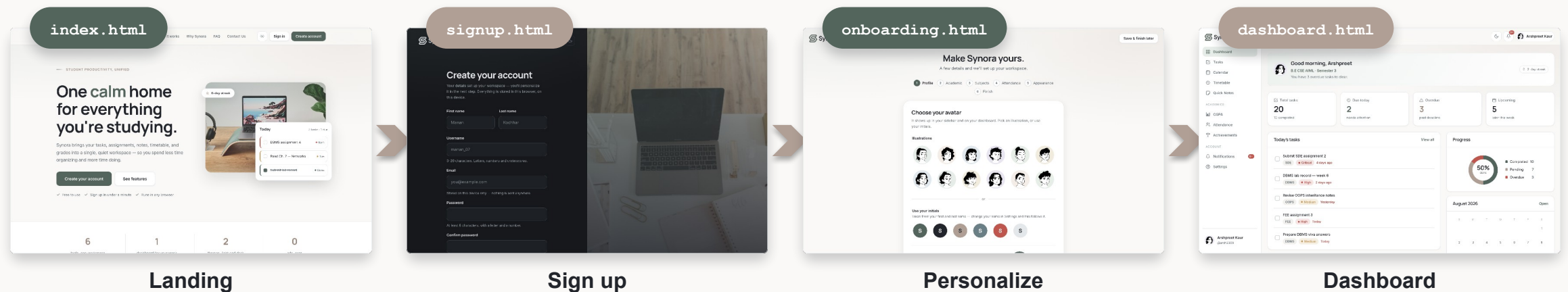
<header class="site-header">
  <nav aria-label="Primary"> ... </nav>
</header>

<main id="main">
  <section class="hero">...</section>
  <section aria-labelledby="features">...
</section>
</main>
<footer class="site-footer">...</footer>
```

Real markup from index.html — trimmed.

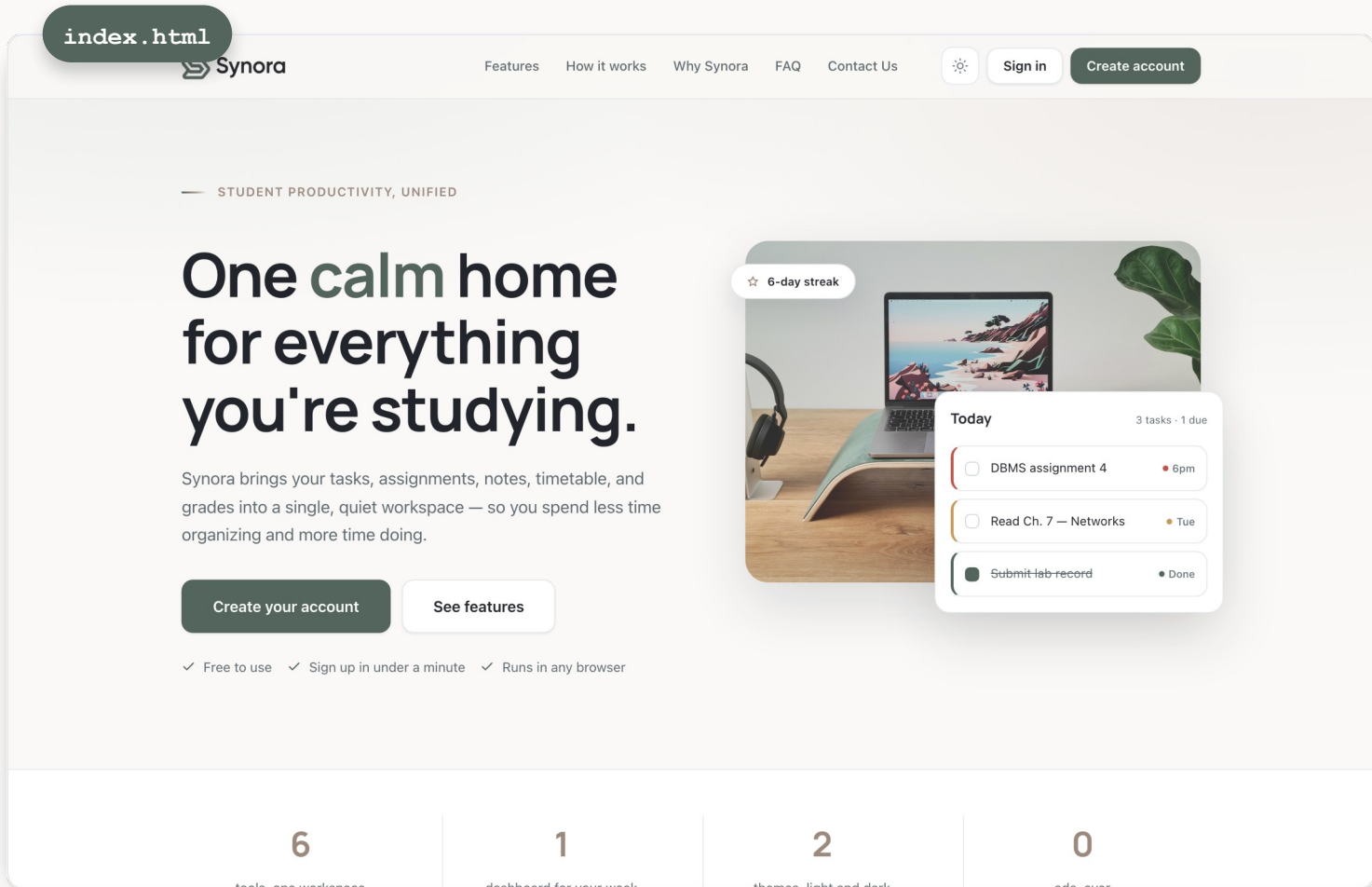
User journey: landing to workspace

Five HTML pages, one continuous flow — structure and links carry the user through.



Each page is a self-contained HTML document; anchors and links move between them, and the same navigation structure repeats so the product feels like one continuous space.

The landing page



Hero, navigation and marketing sections.

-  **Hero**
headline + product preview
-  **Navbar**
centred links + theme + auth
-  **Features / How it works**
semantic sections with IDs
-  **FAQ (CSS-only)**
accordion, no JavaScript
-  **Contact**
a real, validated form
-  **Footer**
links + sitemap

The account flow

signup.html

Synora

Create your account

Your details set up your workspace — you'll personalize it in the next step. Everything is stored in this browser, on this device.

First name Last name

Manan Kochhar

Username

manan_07

3-20 characters. Letters, numbers and underscores.

Email

you@example.com

Stored on this device only — nothing is sent anywhere.

Password

At least 8 characters, with a letter and a number.

Confirm password

One form, the right input type per field.

1. Sign up

first & last name, username, email, password

2. Confirmation

account created, stored on this device

3. Personalize

the onboarding wizard sets up the workspace

4. Dashboard

the populated product

Input types — text, email, password — give the right keyboard and the browser's first check.

Personalization: building the workspace

Synora

Save & finish later

Make Synora yours.

A few details and we'll set up your workspace.

1 Profile 2 Academic 3 Subjects 4 Attendance 5 Appearance 6 Finish

Choose your avatar

It shows up in your sidebar and on your dashboard. Pick an illustration, or use your initials.

Illustrations

or

Use your initials

Taken from your first and last name — change your name in Settings and this follows it.

S S S S S S



Profile

avatar illustration or initials



Academic

program + semester



Subjects

name, credits, track attendance?



Attendance

target percentage

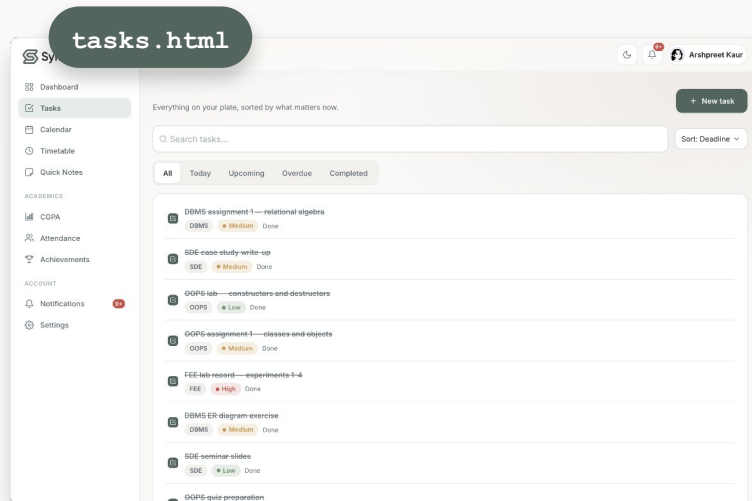


Appearance

light or dark theme

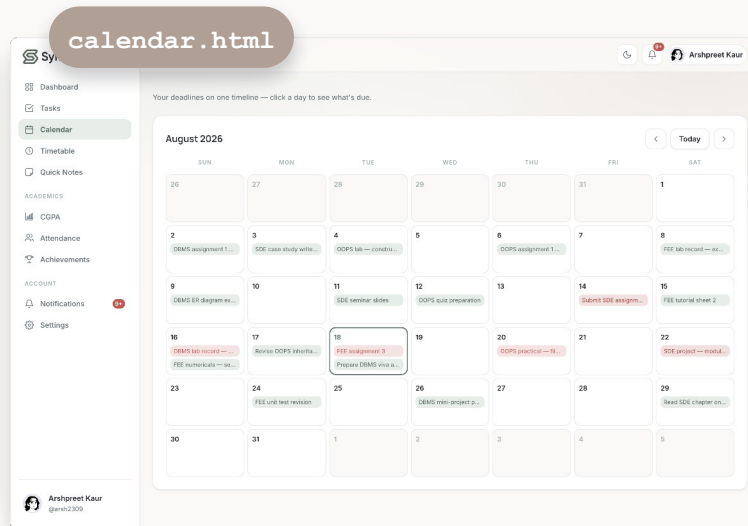
app/onboarding.html — a six-step wizard of HTML form controls.

Feature walkthrough — Productivity



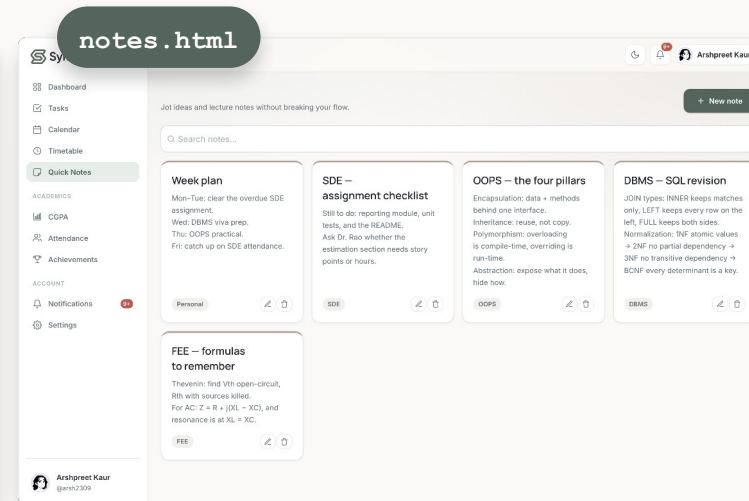
Tasks

Create, prioritise, complete — filtered by today / upcoming / overdue



Calendar

Every deadline on one timeline; click a day to see what's due

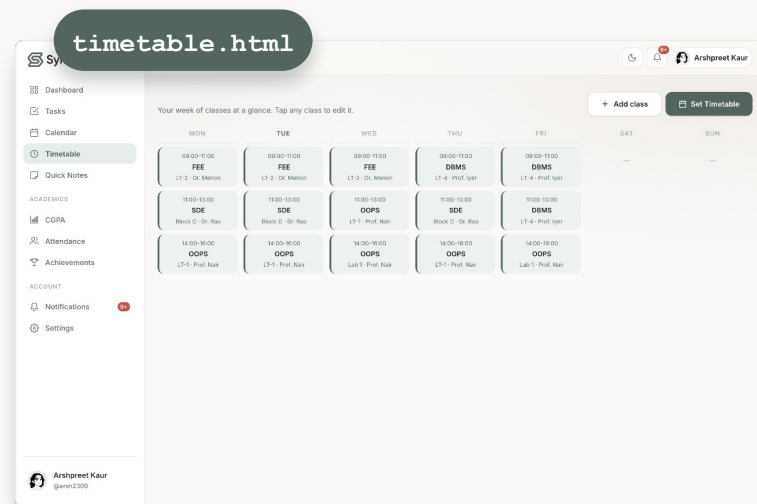


Quick Notes

Jot lecture notes as cards, tagged by subject

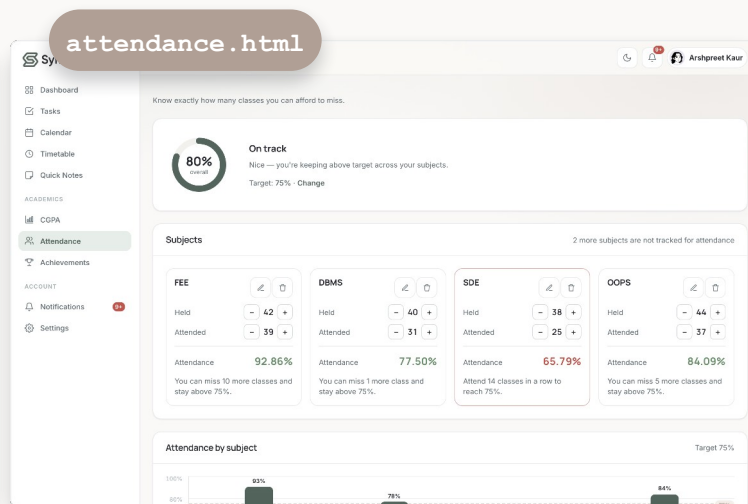
Structured HTML — lists, cards, forms and controls — gives each feature its shape.

Feature walkthrough — Academic tools



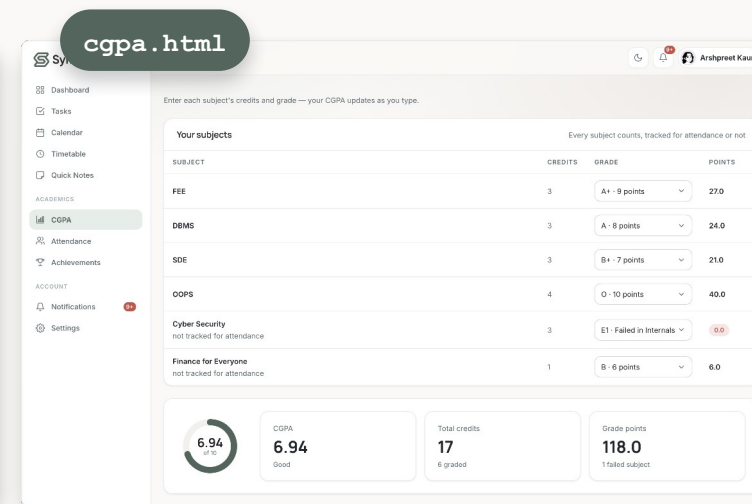
Timetable

Your week of classes as an editable grid



Attendance

How many classes you can still miss, per subject

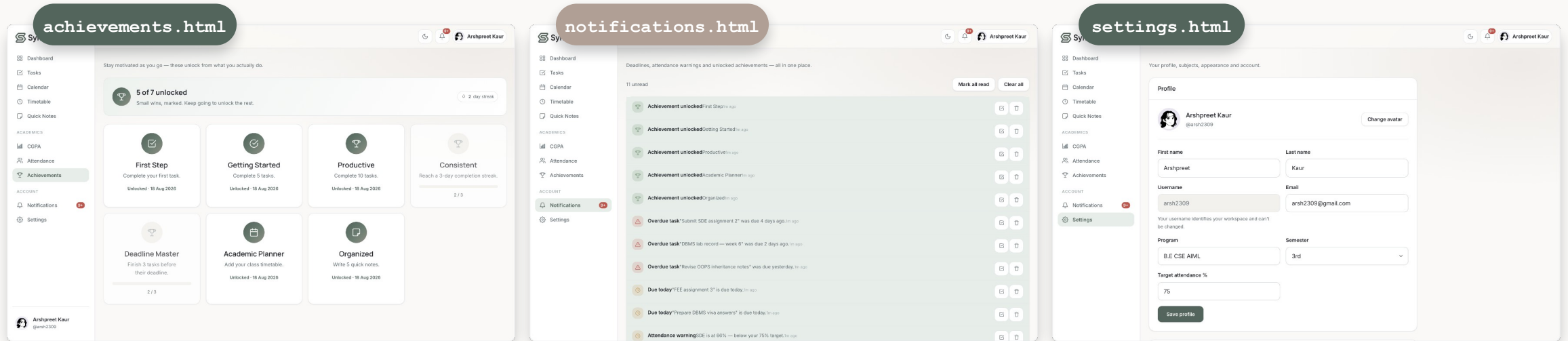


CGPA

Live 10-point GPA as you enter credits and grades

Cards, labels, form controls and chart containers structure the academic data.

Feature walkthrough — Progress & account



Achievements

Small wins unlock as you actually use Synora



Notifications

Deadlines, attendance warnings and unlocks in one place



Settings

Profile, subjects, appearance and account

Status indicators, controls and structured lists — all built from semantic HTML.

Forms & accessibility

settings.html

Synora Settings

Your profile, subjects, appearance and account.

Profile

Arshpreet Kaur
@arsh2309 [Change avatar](#)

First name
Arshpreet

Last name
Kaur

Username
arsh2309
Your username identifies your workspace and can't be changed.

Email
arsh2309@gmail.com

Program
B.E CSE AIML

Semester
3rd

Target attendance %
75

[Save profile](#)

Every field has a real `<label>` and the right input type.



Labelled inputs

16 `<label>` elements — a label focuses its field



ARIA where it helps

91 aria-* attributes — labels, live regions



Right input types

email, number, date, password



Alt text

18 alt attributes; decorative art hidden

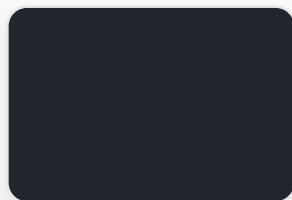


Keyboard-friendly

a skip-link + a `.visually-hidden` pattern

One design system, not many pages

CSS custom properties define the visual language once — 120+ design tokens (85 unique) for colour, type, spacing, radius and shadow, reused across every page so the product feels like one system.



Ink

#22252B



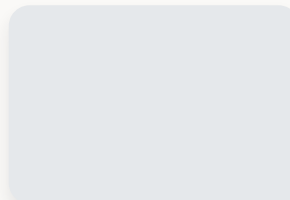
Sage

#53655C



Slate

#A3AEB1



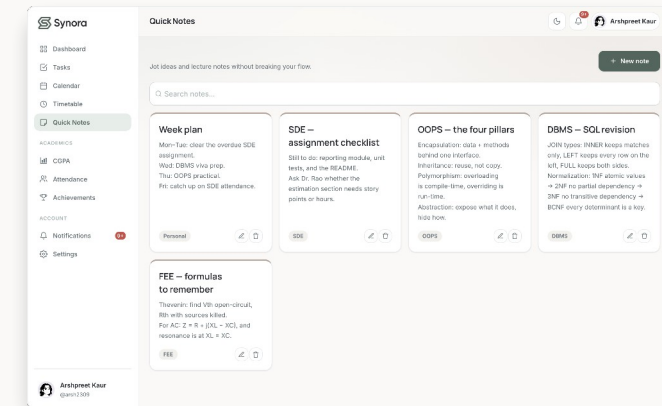
Mist

#E5E8EB



Taupe

#B09F95



Same cards, buttons and spacing tokens on every page.

TYPOGRAPHY

Manrope

headings · Inter for body

REUSED COMPONENTS

`.btn` `.card` `.field` `.chip`

defined once, used everywhere

One file defines everything — tokens.css

Every colour, font, space, radius and shadow is a CSS custom property in a single `:root` block — the one source of truth the whole app reads from.

```
styles/tokens.css

:root {
  --color-bg:      #FAF9F7;
  --color-brand:   #53655C; /* sage */
  --color-text:    #22252B;
  --font-heading:  "Manrope", ...;
  --space-16:      16px;
  --radius-lg:     16px;
  --shadow-sm:     0 1px 3px ...;
}

[data-theme="dark"] { /* same names,
  --color-bg:      #16181C;
  --color-brand:   #8FA697;    new values */
}
```

WHAT IT ORGANISES



Colour

surfaces · text · brand · status



Typography

Manrope + Inter, a type scale



Spacing

4px base — space-4 ... space-128



Radii

sm · md · lg · xl · pill



Elevation

shadow-sm ... shadow-xl

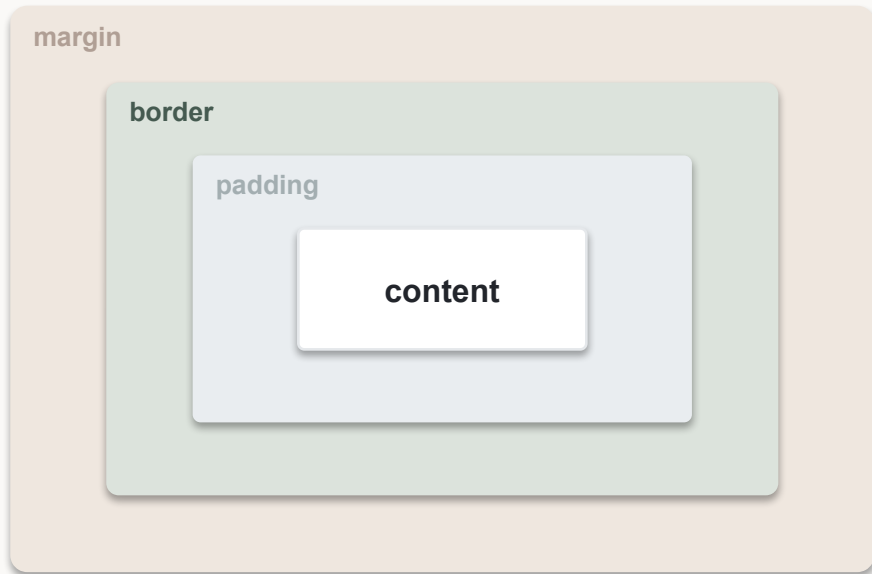


Motion

easing curves + durations

Components never hard-code a value — they read a token. One `:root`, ~85 tokens; dark mode just re-declares the same names.

The box model behind the interface

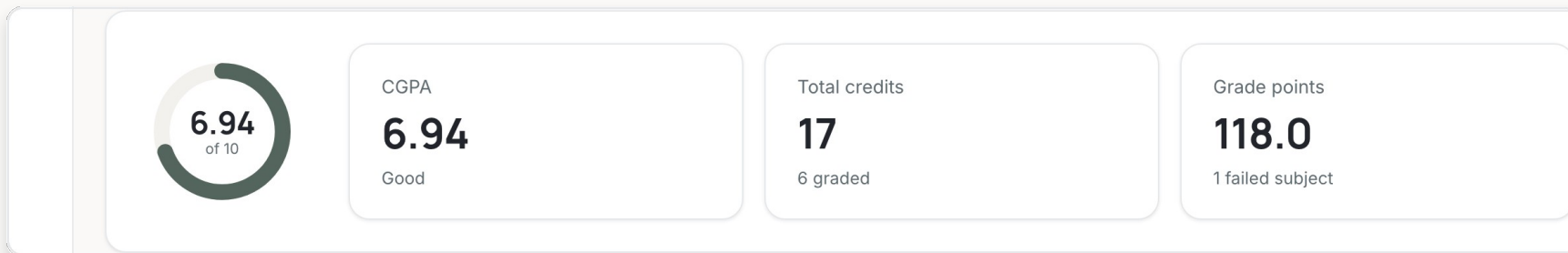


content → padding → border → margin

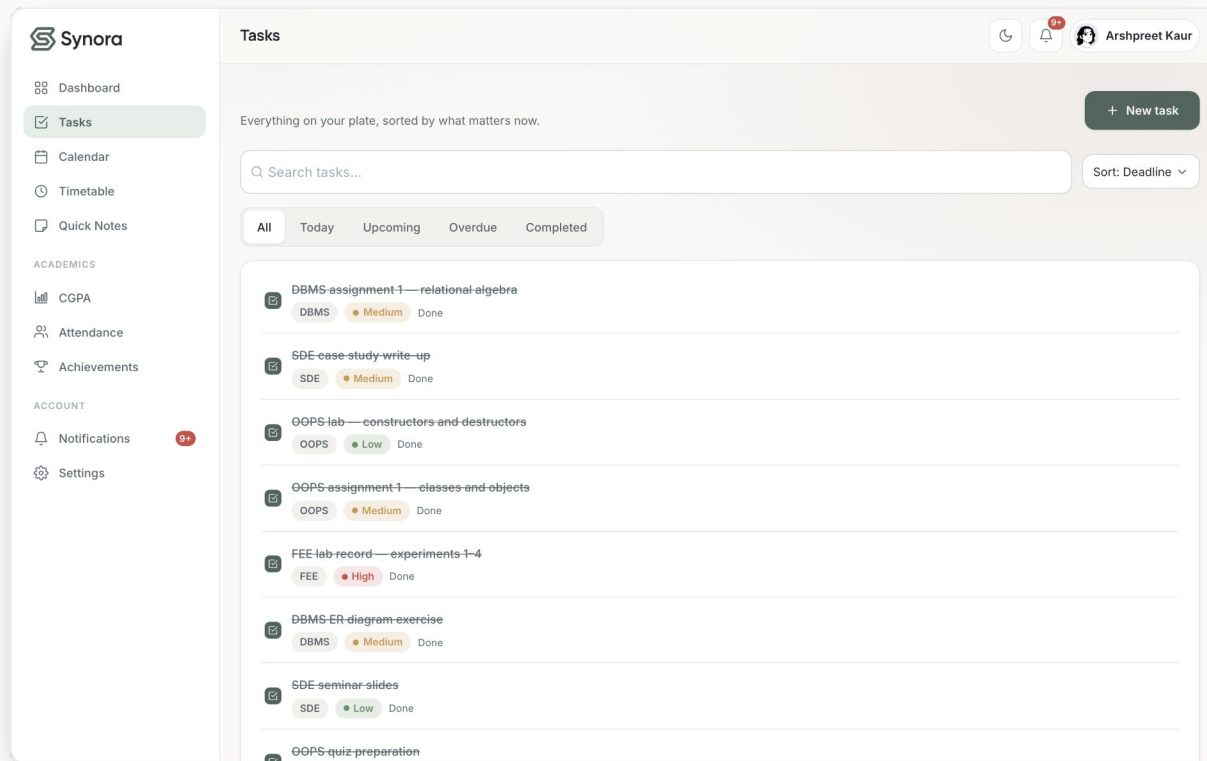
```
styles/app.css — a dashboard stat card

.stat-card {
  padding: 20px 24px;      /* space inside */
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
}
```

Those same tokens build every card — here, the CGPA summary row:



Flexbox — alignment & flow



The Tasks toolbar — search, sort and view controls in a flex row.

One-dimensional layout



display: flex

lay children out in a row



align-items: center

vertically centre them



gap

even spacing without margins

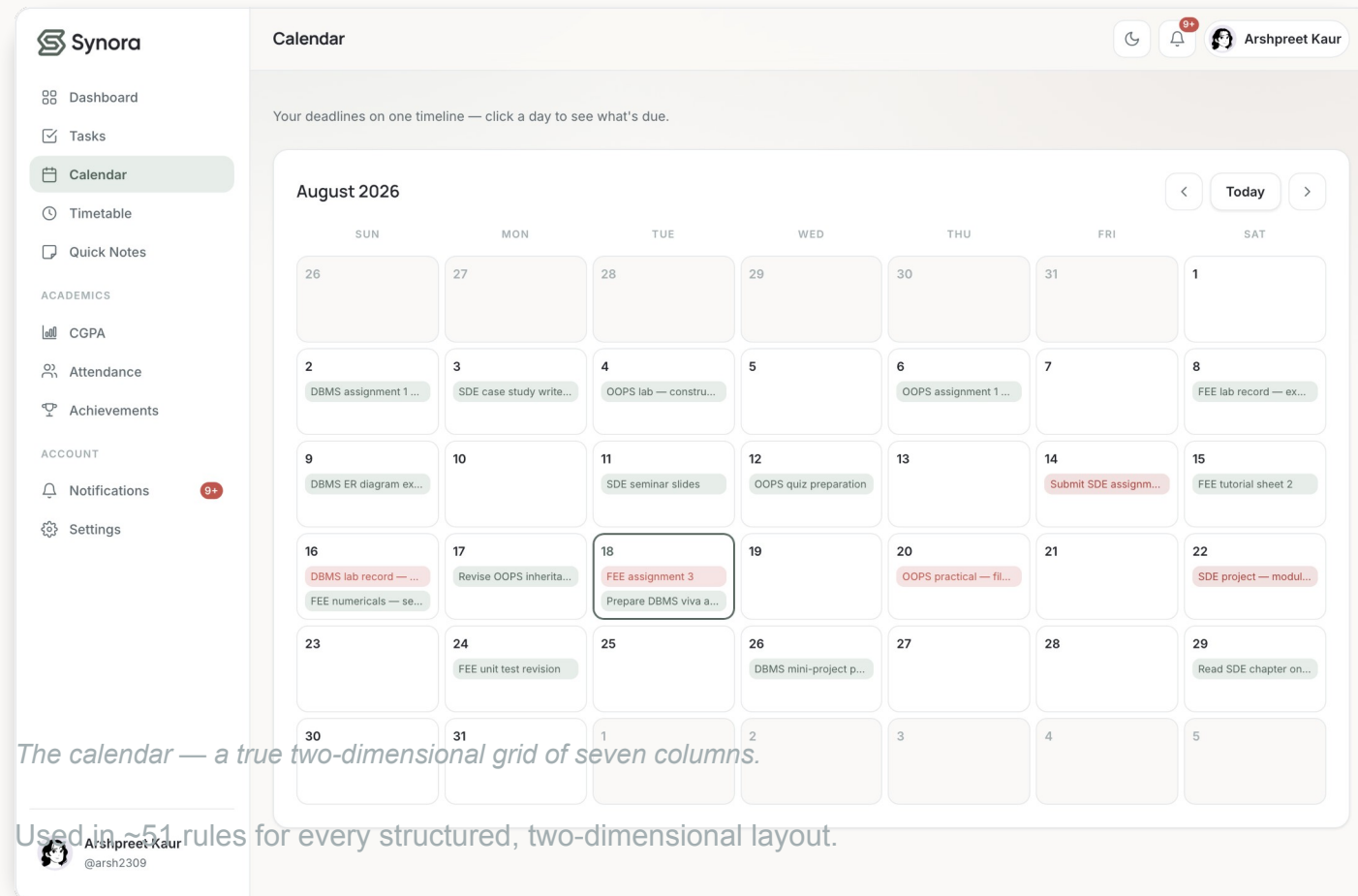


flex-wrap: wrap

fold onto the next line when tight

Used in ~116 rules — top bar, toolbars, button groups, list rows.

Grid — structuring the layouts



Two-dimensional layout



Rows & columns

the calendar is 7 columns × N weeks



Dashboard cards

auto-fitting stat grid



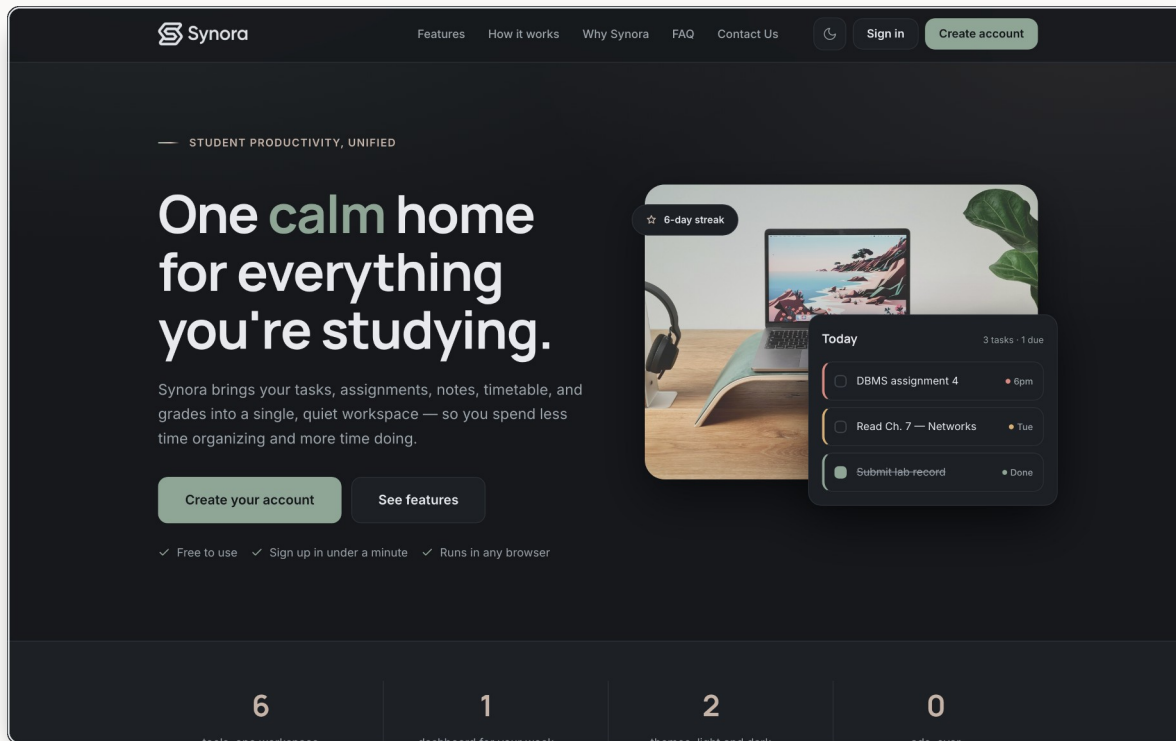
Notes masonry

cards flow into a responsive grid

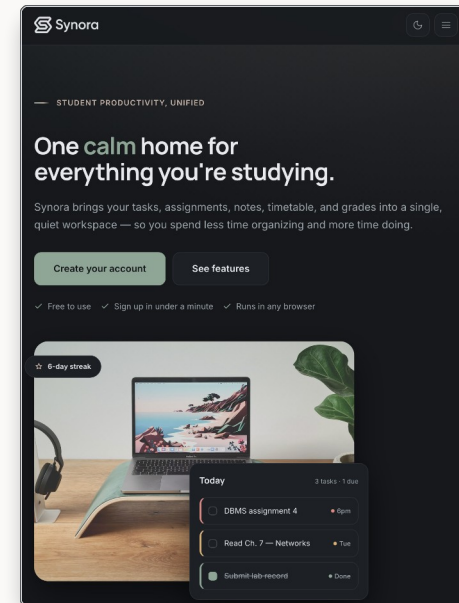
```
styles/app.css

.cal-grid {
  display: grid;
  grid-template-columns:
    repeat(7, minmax(0, 1fr));
}
```

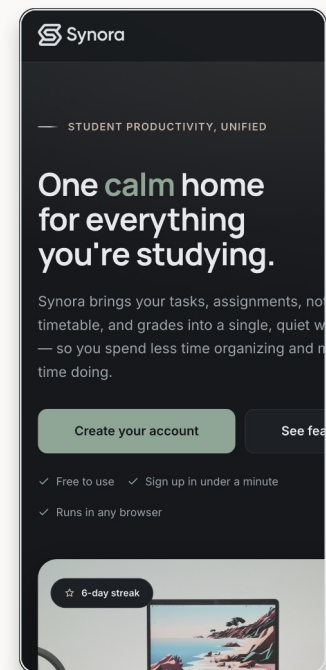
One Synora, every screen



1440px · Desktop



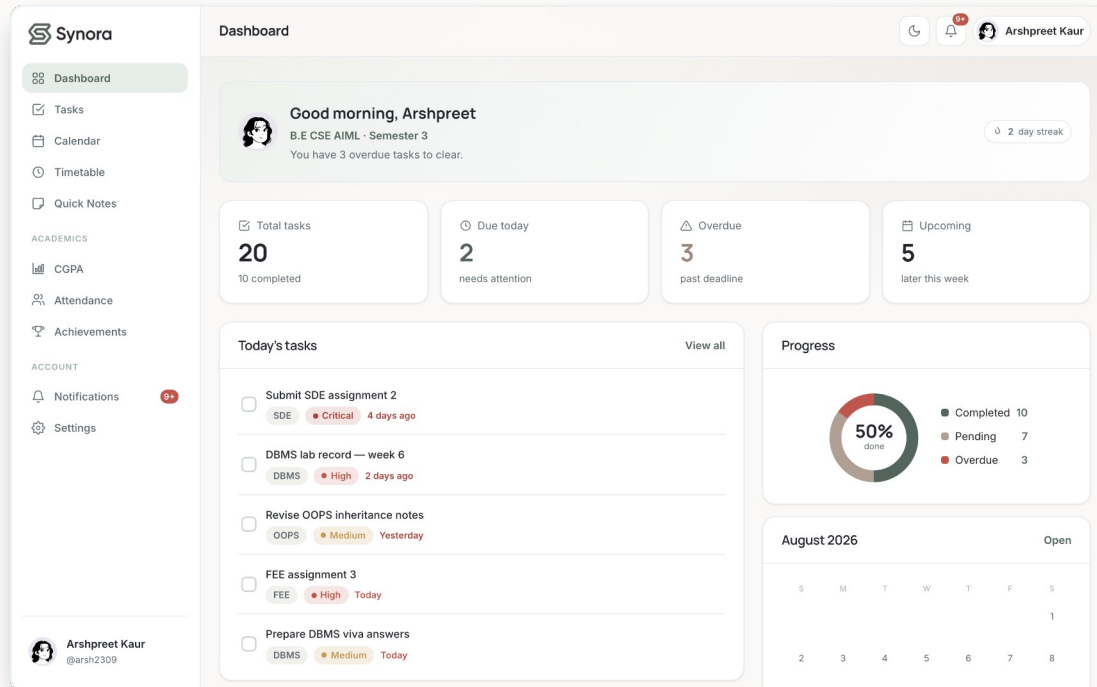
768px · Tablet



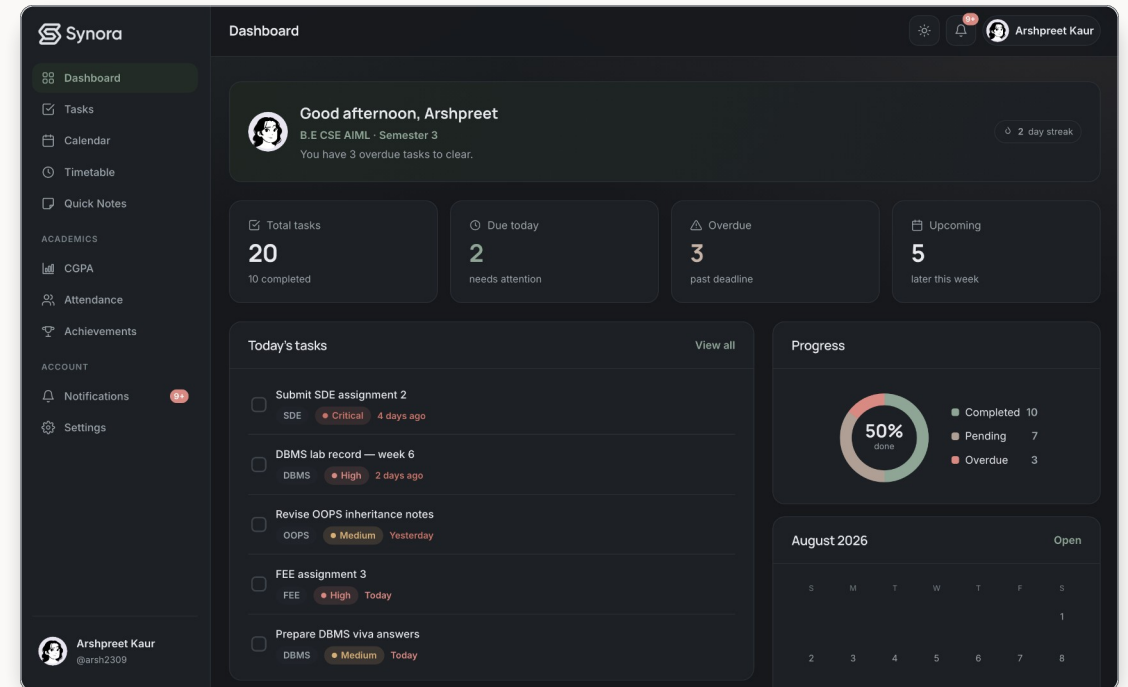
390px · Mobile

Same landing page — Flexbox + Grid reflow, 23 media-query blocks adapt down, and the nav collapses to a CSS-only menu.

Dark mode & theming



Light



Dark

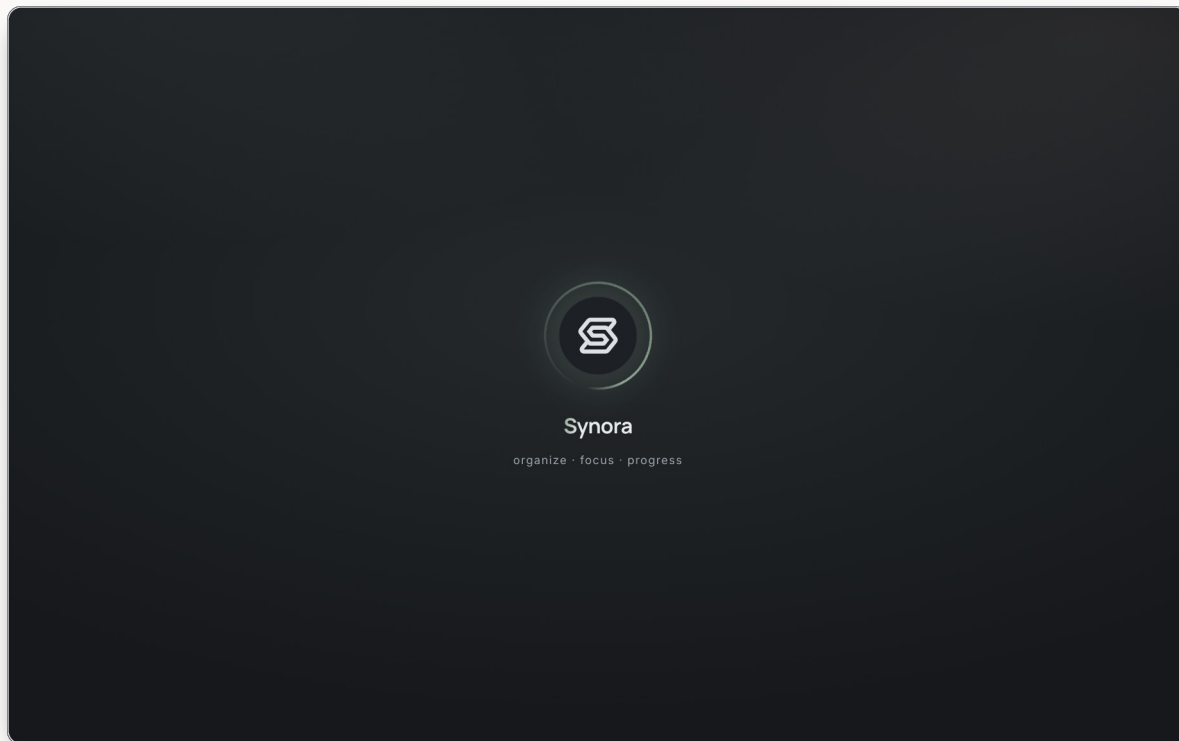
```
scripts/theme.js

var saved = localStorage.getItem("synora-theme");
root.setAttribute("data-theme", theme); // CSS reacts
```

Same tokens, two themes.

`data-theme` on `<html>` swaps every token; the choice is saved to `localStorage` and applied before first paint — no flash.

Before Synora appears



The Synora splash — logo, ring and wordmark.

THE FLOW

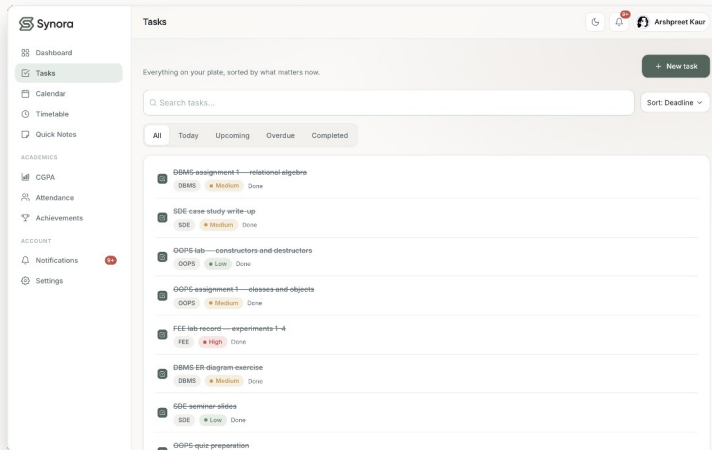
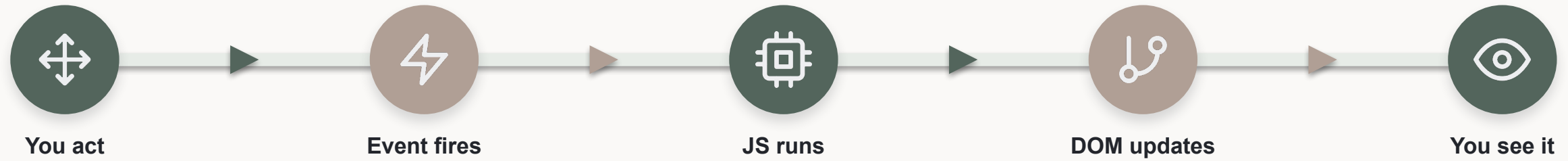
-  **Splash**
logo, ring & wordmark animate in
-  **Handover**
the splash fades and lifts away
-  **Landing**
the hero arrives underneath

HOW IT WORKS

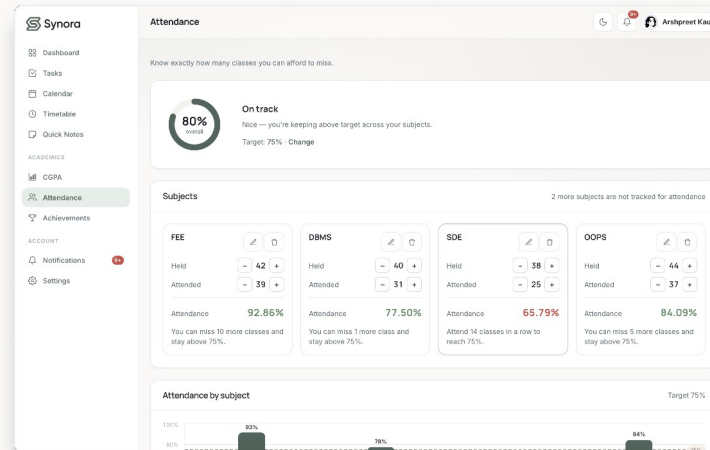
- CSS @keyframes** — pure-CSS animation
- JavaScript timing** — shows $\geq 2000\text{ms}$, then `loader.remove()`
- Reduced motion** — honoured — 600ms, no animation

Making Synora interactive

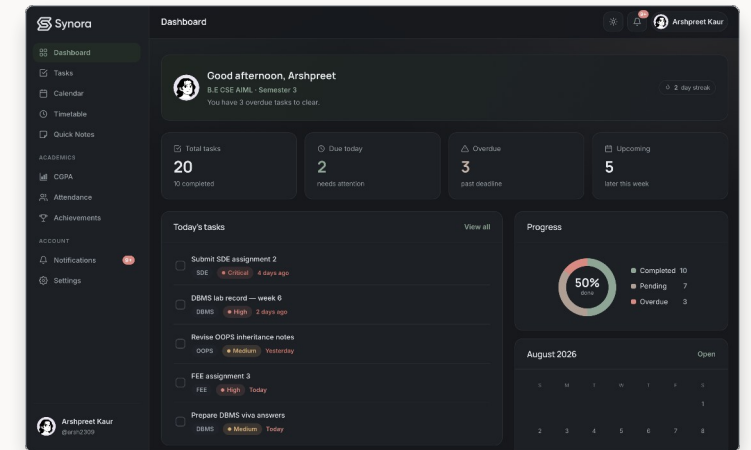
HTML and CSS render the page — JavaScript is what makes it respond. Every click, keystroke and toggle runs code that updates the screen:



Tick a task → it completes and the counts drop



Tap +/- → every percentage recomputes live



Toggle the theme → the whole app repaints

Fundamentals, mapped to Synora



Variables

hold application state

```
state = { view, sort, query }
```



Functions

reusable operations

```
render() · toggleComplete()
```



Arrays

lists of things

```
tasks · subjects ·  
notifications
```



Objects

structured records

```
{ id, title, priority,  
  deadline }
```



Loops

render each item

```
.map() · .filter()  
· .forEach()
```

ALL FIVE, ONE FUNCTION

```
tasks.js  
  
function filtered(){  
  items = all().slice();  
  items = items.filter(fn)  
    .sort(byDate);  
  return items;  
}
```

Variables, functions, arrays, objects and loops —
together in a few lines.

The DOM — bridge between JS and UI

The page is a live tree of nodes:

```
document
├── main.page
│   ├── section.toolbar
│   │   └── input[data-search]
│   └── div[data-list]
│       └── div.trow           one node per task
```

SELECTORS ACTUALLY USED

`querySelector()` ×161

`querySelectorAll()` ×23

`getElementById()` ×9

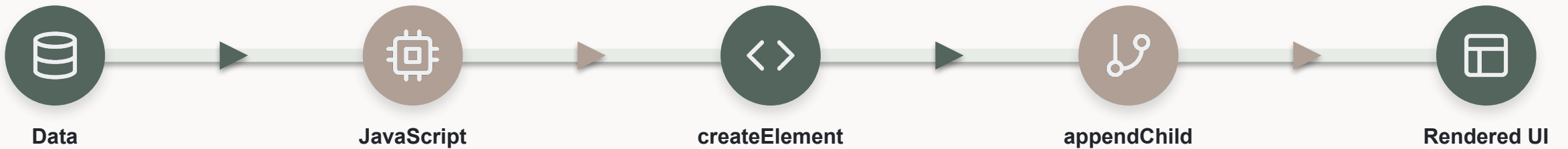
JavaScript selects, reads and writes

```
scripts/tasks.js

var root = document.querySelector("[data-tasks-page]");
root.querySelectorAll("[data-view]").forEach(...);
el.classList.toggle("is-active", on); // write CSS
el.textContent = value;               // write text
```

Select an element, then read or change its text, its classes (`classList` toggled ~35×) or its attributes — and the browser repaints instantly. The DOM is the object tree JavaScript reaches through to change what you see.

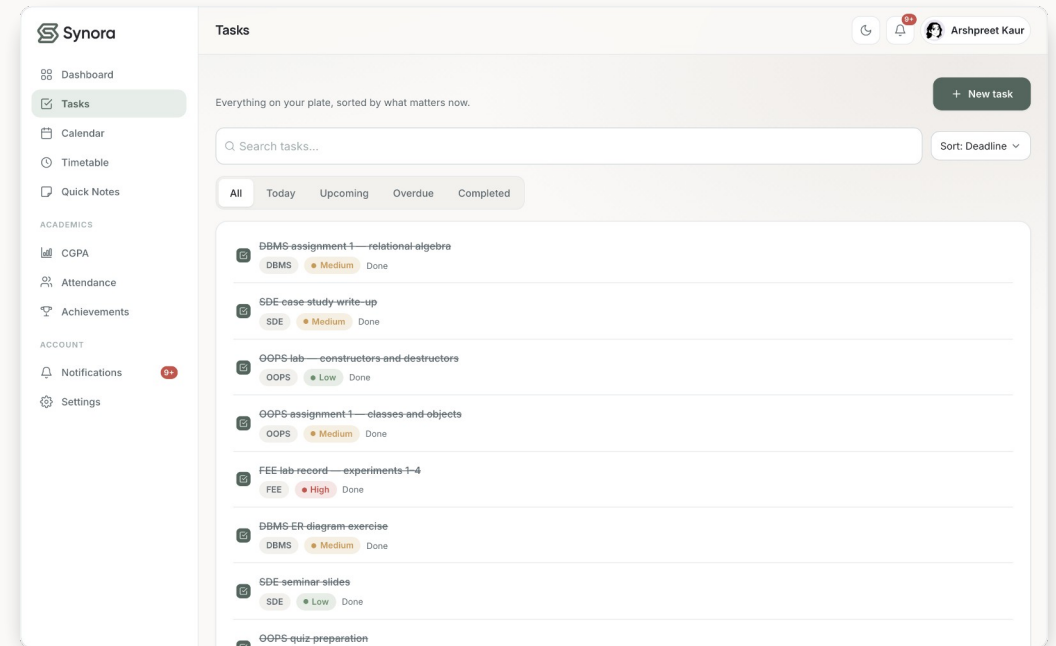
Building the interface dynamically



```
scripts/app.js — a node, born and removed

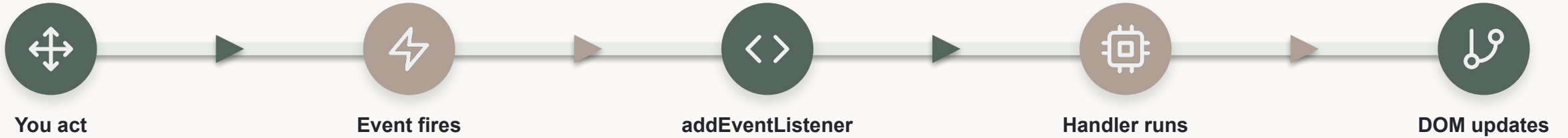
var el = document.createElement("div"); // 1 create
el.className = "toast";
el.setAttribute("role", "status"); // 2 config
region.appendChild(el); // 3 insert
... el.remove(); // 4 delete
```

Every task row is built from data and rendered into the list the same way —
`createElement` × 18, `appendChild` × 11, `element.remove()` × 17.



Tasks — a list rendered node by node.

Event handling & delegation



```
scripts/tasks.js - event delegation
```

```
// ONE listener handles every task row (delegation)
container.addEventListener("click", function(e) {
  var t = e.target.closest("[data-toggle]");
  if (t) { toggleComplete(t...); rerender(); }
});
```

- ✓ **addEventListener** used ~93× across the app
- ✓ **Event delegation** used — `.closest()` in ~28 handlers
- ✓ **Bubbling** used — clicks bubble to the parent list
- ✓ **preventDefault** used — forms & links (~11)
- ⚠ **Explicit capturing** *concept only — not a pattern we use*

Rows re-render constantly, so one durable listener on the parent beats binding one per button.

Forms, validation & interaction

Sign-in / sign-up validate on submit and clear errors as you type.



HTML attributes

required, type=email, minlength — the browser's first check



JavaScript on submit

preventDefault(), trim, regex, friendly inline errors



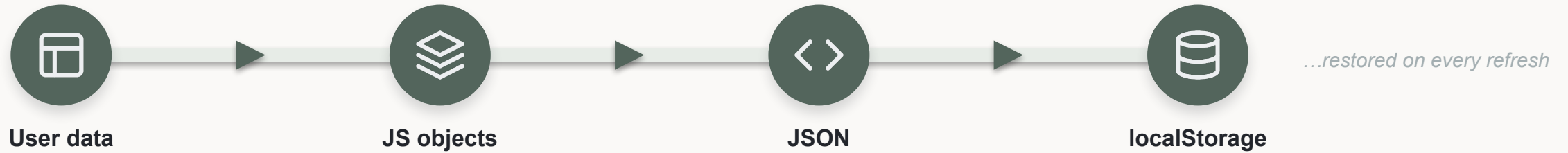
Live error clearing

the message clears the moment the field is fixed

```
scripts/auth.js

form.addEventListener("submit", function(e) {
  e.preventDefault(); // stop native submit
  if (!EMAIL_RE.test(email))
    setError("email", "Check your email.");
});
```

Data & persistence



```
scripts/storage.js

store.set("tasks", tasks);
  → localStorage.setItem(key, JSON.stringify(v));

store.get("tasks")
  → JSON.parse(localStorage.getItem(key));
```

WHAT IS STORED — PER USER

synora-users · synora-session · synora-theme

synora:<user>:subjects (source of truth)

...:tasks · notes · timetable · attendance

...:cgpa · achievements · notifications · profile

Every account gets its own namespaced workspace. Limitation: data is per-browser, not synced across devices.

The logic we coded

The academic numbers aren't stored — JavaScript computes them live from your subjects. The real formulas behind the screens:



Attendance %

```
attended / held × 100
```

how much of each subject you've attended



CGPA

```
 $\Sigma(\text{credits} \times \text{grade point}) / \Sigma(\text{credits})$ 
```

weighted by each subject's credits



Classes you can safely miss

```
floor(attended × 100 / target) - held
```

when you're already above target



Classes to attend in a row

```
ceil((target × held - 100 × attended) / (100 - target))
```

to climb back above target

Subjects are the source of truth — attendance, CGPA, timetable and tasks all reference a subject by id, in each user's own workspace.

● SUMMARY

What we built beyond the syllabus

SYLLABUS

HTML5

CSS3

JavaScript

DOM

Events



Authentication



Avatar system



Calendar



Timetable



CGPA logic



Notifications



Personalized onboarding



Task management



Quick Notes



Attendance logic



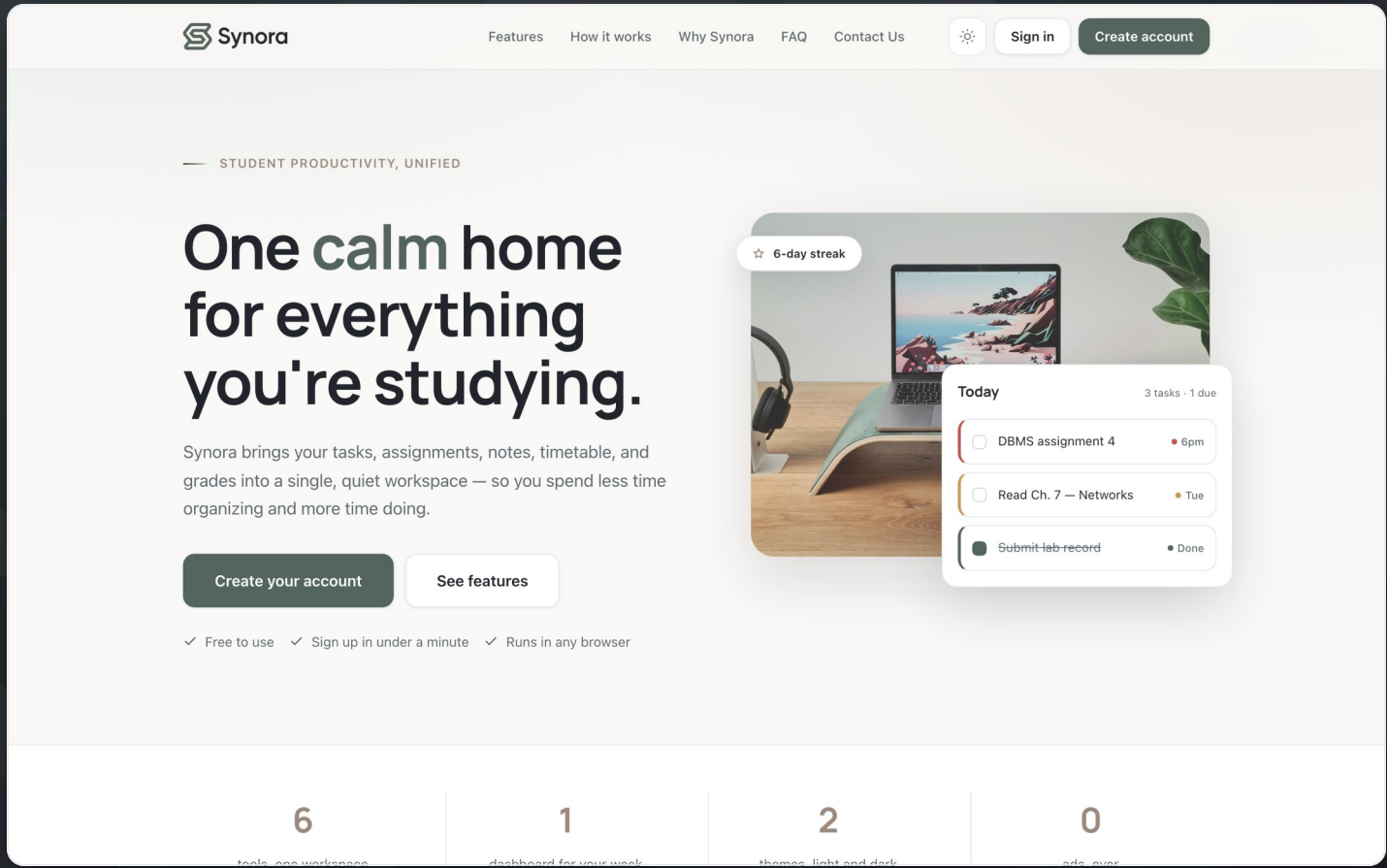
Achievements



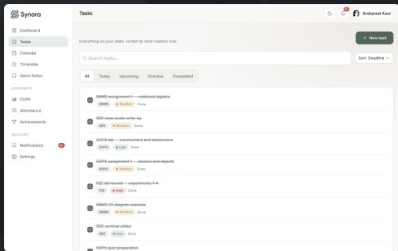
Light / dark mode

THE PRODUCT

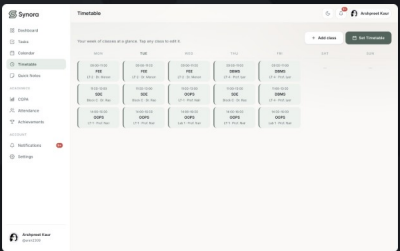
Synora in one view



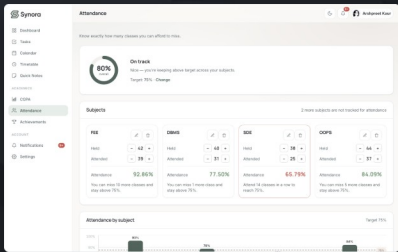
Landing



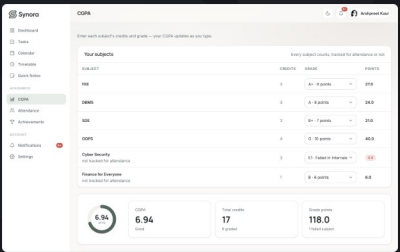
Tasks



Timetable



Attendance



CGPA



Thank you.

Synora — one calm home for everything you're studying.

SUBMITTED BY

Manan Kochhar · Arshpreet Kaur · Atharv Bains



Scan for the live demo

`synora.vercel.app`